Christian Posta

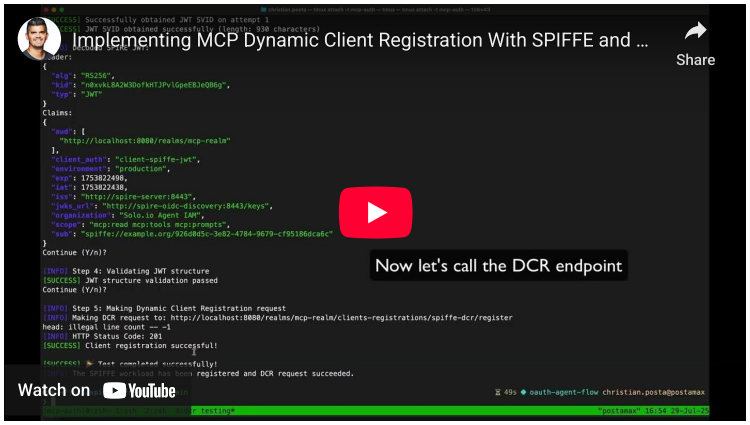
Global Field CTO at solo.io, author 'Istio in Action', 'AI Gateways in the Enterprise' and other books. He is known for being an architect, speaker, blogger and contributor to AI and infrastructure open-source projects.

LinkedIn
Twitter
Github
Stackoverflow

Implementing MCP Dynamic Client Registration With SPIFFE and Keycloak

The MCP Authorization spec recommends using OAuth Dynamic Client Registration (DCR) for registering MCP clients with MCP servers. More specifically, it suggests using anonymous DCR: meaning any client should be able to discover how to register itself and dynamically obtain an OAuth client without any prior credentials. In a recent blog post, I explored why this model can be problematic in enterprise environments where anonymous registration is often restricted or outright disabled. In this blog, we'll look at how SPIFFE can be used for dynamic client registration.

TL;DR If you want to see a quick demo of this working:



There are other options than anonymous DCR. The RFC 7591 spec on Dynamic Client Registration talks about:

- **Manual client registration**
- **Initial Access Token (IAT)**
- **Software Statements**

Most enterprises are familiar with manually registering an OAuth client. This involves the administrator doing this (or some automated workflow) and issuing client IDs and client secrets. Care must be taken to share the ID and secret. For initial access tokens (IATs), the Authorization Server (AS) administrators issue a token ahead of time that can be used to call the registration endpoints and register an OAuth client dynamically. There needs to be some coordination here to safely get the IAT to the MCP client so that it can register a client. This way, only approved MCP clients would be able to register an OAuth client, and this list can be governed.

Another approach is to use a cryptographically signed / trusted token with "software statements" which assert facts about the client. These "software statements" can be trusted by the AS and then used to register the OAuth client. For example, a provider creating a JWT with software statements to be used for DCR might look like this:

```
// Software vendor creates signed statement
const softwareStatement = jwt.sign({
  iss: 'https://software-vendor.example.com',
  sub: 'mobile-banking-app-v2.1',
  aud: 'https://auth-server.example.com',
  software_id: 'banking-app-uuid-12345',
  software_version: '2.1.0',
  software_client_name: 'Official Banking App',
  software_client_uri: 'https://bank.example.com/app',
  software_redirect_uris: ['https://bank.example.com/callback']
});
```

Then an MCP client can call the OAuth registration with the following:

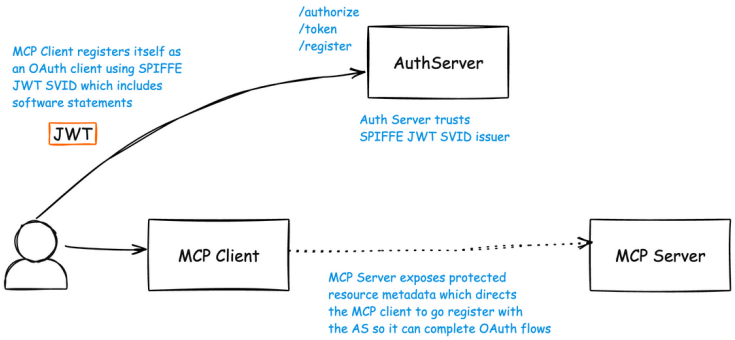
```
POST /register HTTP/1.1
Host: auth.example.com
Content-Type: application/json

{
  "software_statement": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "client_name": "Official Banking App",
  "redirect_uris": ["https://bank.example.com/callback"]
}
```

Software Statements with SPIFFE SVIDs

SPIFFE is a specification and commonly used standard for workload identities which can be used for agent and MCP identity. SPIFFE helps to get rid of static secrets, passwords, and other long-lived credentials and instead relies on runtime attestation and issuance of a type of cryptographically verifiable credential called an SPIFFE Verifiable Identity Document (SVID). SPIRE is a popular implementation of SPIFFE.

Recently, a Internet Draft with the IETF was created by Pieter Kasselman et. al. describing an approach using software statements with SPIFFE/SPIRE to dynamically register an OAuth client with an Authorization Server. This approach eliminates the need for anonymous DCR, IATs, or manual registration. This also eliminates the need for any static OAuth client credentials/secrets/passwords. We can leverage existing attestation and identity mechanisms (derived from SPIFFE/SPIRE) to register an OAuth client for our MCP connectivity.



I've recently implemented this draft spec in some working examples I've been exploring and would like to share how it all works.

Extending SPIRE and Keycloak to support SPIFFE based DCR

To get this POC to work, we need to extend both Keycloak and SPIRE as neither support this out of the box. However, both have very nice plugin models making extensions fairly straight forward. We will start with extending Keycloak. You can follow along in this GitHub repo to see the code.

Extending Keycloak

Keycloak is written in Java and has a nice "Service Provider Interface" model for extending many parts of Keycloak. For Dynamic Client Registration (DCR), we need to implement the ClientRegistrationProviderFactory interface to create a custom DCR endpoint that understands SPIFFE software statements.

Core SPI Architecture

The extension consists of three main components:

- Factory Class - Tells Keycloak how to create our provider
- Provider Class - Implements the actual DCR logic with SPIFFE support
- Service Registration - Makes Keycloak discover our extension

You can see the Factory Class and Service Registration in the GitHub repo. The meat of the extension is the SpiffeDcrProvider

Specifically, we use a class called `SpiffeSoftwareStatementValidator` to inspect the JWT for key claims. These claims act as trusted "software statements" that Keycloak uses to register the client. The validator checks that the issuer matches a trusted SPIRE trust domain, that the subject is a valid SPIFFE ID, and that the `client_auth` claim is present. This last claim determines how the client will authenticate, allowing us to configure the appropriate OAuth client authentication mechanism. For example, we could use SPIFFE JWT SVIDs for client authentication, though we'll cover that in a separate post.

```
public class SpiffeSoftwareStatementValidator {

    public SpiffeValidationResult validateSoftwareStatement(String jwt) {
        try {
            // Parse the JWT software statement
            SignedJWT signedJWT = SignedJWT.parse(jwt);
            JWTClaimsSet claims = signedJWT.getJWTClaimsSet();

            // Validate SPIFFE-specific claims
            String spiffeId = claims.getSubject();
            if (!isValidSpiffeId(spiffeId)) {
                return SpiffeValidationResult.invalid("Invalid SPIFFE ID format");
            }

            // Validate trust domain matches realm configuration
            String trustDomain = extractTrustDomain(spiffeId);
            if (!isValidTrustDomain(trustDomain)) {
                return SpiffeValidationResult.invalid("Trust domain not allowed");
            }

            // Fetch SPIRE server's JWKS for signature verification
            JWKSet jwkSet = fetchSpireJwks();
            if (!verifySignature(signedJWT, jwkSet)) {
                return SpiffeValidationResult.invalid("Invalid JWT signature");
            }

            // Validate required SPIFFE claims
            if (!"client-spiffe-jwt".equals(claims.getStringClaim("client_auth"))) {
                return SpiffeValidationResult.invalid("Invalid client_auth claim");
            }

            return SpiffeValidationResult.valid(claims);

        } catch (Exception e) {
            return SpiffeValidationResult.invalid("JWT parsing failed: " + e.getMessage());
        }
    }
}
```

With this SPI implemented, we can load it into Keycloak at runtime. Here's an example doing so with Docker Compose:

```
services:
  keycloak-idp:
    image: quay.io/keycloak/keycloak:26.2.5
    environment:
      KC_HEALTH_ENABLED: "true"
      KEYCLOAK_ADMIN: admin
      KEYCLOAK_ADMIN_PASSWORD: admin
    ports:
      - "8080:8080"
    volumes:
      - ./spiffe-dcr-spi-1.0.0.jar:/opt/keycloak/providers/spiffe-dcr-spi-1.0.0.jar:ro
    command: start-dev
    networks:
      - keycloak-shared-network
```

This JAR file will automatically get picked up by Keycloak, and make available a new DCR option. Here's an example of calling this endpoint. It can be called from an MCP client to

register itself to an MCP Server's Authorization Server (Keycloak):

```
# Service obtains its SPIFFE SVID JWT from SPIRE agent
SPIFFE_JWT=$(curl unix:/tmp/spire-agent/public/api.sock/svid/jwt)

# Self-register as OAuth client
curl -X POST \
  "https://keycloak.example.com/realms/production/clients-registrations/spiffe-dcr/register"
  -H "Content-Type: application/json" \
  -d "{
    \"software_statement\": \"$SPIFFE_JWT\",
    \"client_name\": \"Payment Service\",
    \"grant_types\": [\"client_credentials\"]
  }"
```

This gives us the foundation for dynamically registering a client with SPIFFE JWT SVIDs in Keycloak. But SPIRE does not natively support software statements for JWT SVIDs. Let's see how to do that.

Extending SPIRE

We will need to configure SPIRE to create JWTs with software statements. SPIRE is written in `golang` and can be extended with go-plugins using the spire-plugin-sdk. SPIRE has the concept of a "credential composer" plugin which can be used to enrich SVIDs before they are signed and returned to the workload through the workload API. You can see the full implementation at the GitHub repo.

We can implement the software statements in the `plugin.go` code:

```
// ComposeWorkloadJWTSVID adds software statement claims to JWT SVIDs
func (p *Plugin) ComposeWorkloadJWTSVID(ctx context.Context, req *credentialcomposerv1.Compos

        if req.Attributes.Claims == nil {
                req.Attributes.Claims = &structpb.Struct{
                        Fields: make(map[string]*structpb.Value),
                }
        }

        // Add jwks_url claim
        if config.JWKSUrl != "" {
                req.Attributes.Claims.Fields["jwks_url"] = structpb.NewStringValue(config.JWK
        }

        // Add client_auth claim
        if config.ClientAuth != "" {
                req.Attributes.Claims.Fields["client_auth"] = structpb.NewStringValue(config.
        }
```

We can load this into the SPIRE server based on the following Docker compose file

```
services:
  spire-server:
    image: ghcr.io/spiffe/spire-server:1.12.4
    container_name: spire-server
    ports:
      - "18081:8081"
    volumes:
      - ./spire-software-statements-linux:/opt/spire/plugins/spire-software-statements:ro
    command: ["-config", "/etc/spire/server/server.conf"]
    networks:
      - keycloak_keycloak-shared-network
```

Then we can configure the SPIRE server (in server.conf) with the following:

```
// Config holds the plugin configuration
CredentialComposer "software_statements" {
```

```
    plugin_cmd = "/opt/spire/plugins/spire-software-statements"
    plugin_checksum = "0b19c7f1ad1b80d0d7494f9e123cc89b41225f7d39784342b3be3cffb8e07985"
    plugin_data = {
        jwks_url = "http://spire-oidc-discovery:8443/keys"
        client_auth = "client-spiffe-jwt"
        allow_insecure_urls = true  # Enable HTTP for testing
        # Optional: Additional claims
        additional_claims = {
            "scope" = "mcp:read mcp:tools mcp:prompts"
            "organization" = "Solo.io Agent IAM"
            "environment" = "production"
        }
    }
}
```

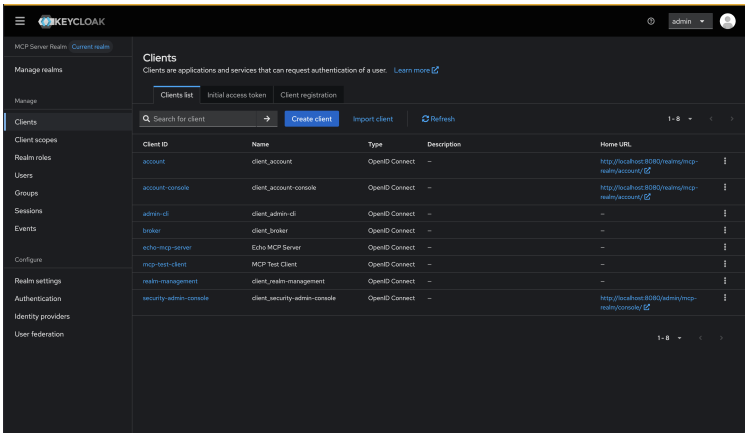
With this piece in place, we can test our DCR using SPIFFE!

Dynamically registering an OAuth Client with SPIFFE JWT SVID

We will start keycloak with our DCR extension. We should see a log statement in the server similar to this to tell us the SPI was loaded correctly:

```
keycloak-idp-1  | 2025-07-29 02:03:09,283 WARN  [org.keycloak.services] (build-38) KC-SERVICES
(com.yourcompany.keycloak.spiffe.dcr.SpiffeDcrProviderFactory) is implementing the internal SPI
client-registration. This SPI is internal and may change without notice
```

When we login to Keycloak, we should see whatever OAuth clients that have been configured manually:



For our example, we will register a sample MCP client workload in SPIRE. This is a very basic registration with a UUID representing the workload/MCP client. SPIRE has sophisticated attestation plugins to verify the workload but that's outside the scope of this blog.

```
Entry ID         : f8260564-1a48-4d65-b1df-86d9cfdd500a
SPIFFE ID        : spiffe://example.org/6e4ac5c5-41a7-45a2-a8d3-e9d2b45ca12b
Parent ID        : spiffe://example.org/agent
Revision         : 0
X509-SVID TTL    : default
JWT-SVID TTL     : 60
Selector         : unix:uid:0
```

Once the workload is registered, we can request a JWT SVID for this workload. It would look like this:

```
{
  "aud": [
    "http://localhost:8080/realms/mcp-realm"
  ],
  "client_auth": "client-spiffe-jwt",
  "environment": "production",
  "exp": 1753755396,
```

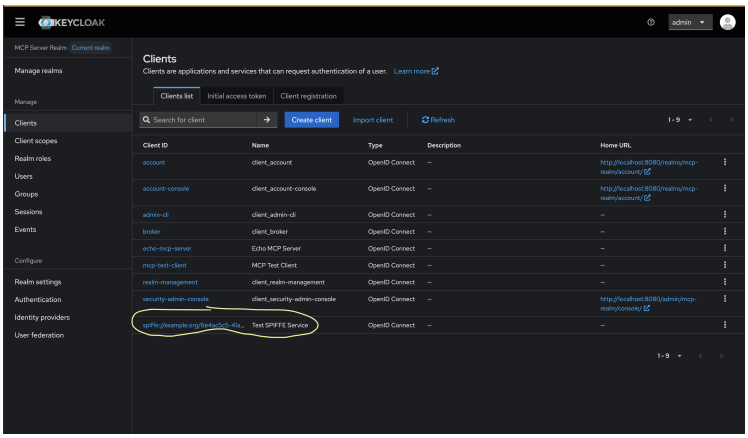
```
  "iat": 1753755336,
  "iss": "http://spire-server:8443",
  "jwks_url": "http://spire-oidc-discovery:8443/keys",
  "organization": "Solo.io Agent IAM",
  "scope": "mcp:read mcp:tools mcp:prompts",
  "sub": "spiffe://example.org/6e4ac5c5-41a7-45a2-a8d3-e9d2b45ca12b"
}
```

Note that the `sub` claim is the SPIFFE ID of the workload we previously registered `spiffe://example.org/d01b3a4b-2c4e-42c1-a1fa-e39790314b9d` and the correct software statements are there, specifically `client_auth` and `jwks_url`. Lastly, note that the correct `aud` is used here, specifically the Keycloak IdP.

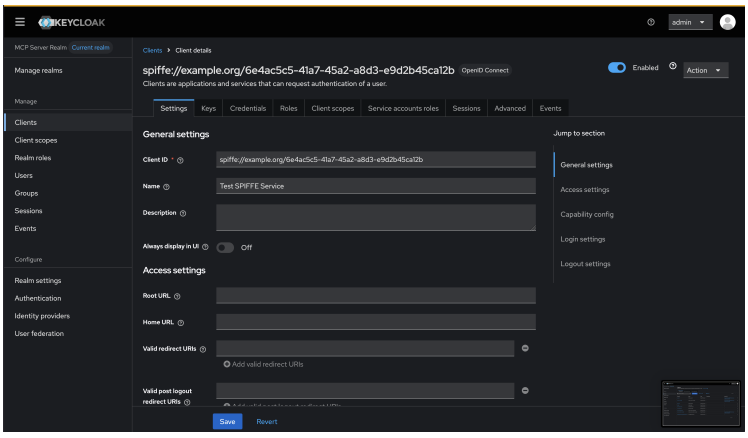
Here's an example request to the SPIFFE Keycloak DCR:

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d "{
    \"software_statement\": \"$JWT_SVID\",
    \"client_name\": \"$CLIENT_NAME\",
    \"grant_types\": [\"client_credentials\"],
    \"scope\": \"spiffe:workload\"
  }" \
  "$KEYCLOAK_URL/realms/$REALM/clients-registrations/spiffe-dcr/register"
```

Once successfully registered, the new OAuth / MCP client should show up in the Keycloak Admin portal:



Clicking into the client, you can see more details:



We can continue to fine tune the client settings and configuration by tuning the software statements

Wrapping up

Dynamic Client Registration for MCP servers is a hot topic, especially in enterprise environments. We can offload the hard part of verifying workloads and issuing identity to a system like SPIFFE/SPIRE and then build on it as we leverage OAuth for user flows. MCP Authorization heavily utilizes OAuth and this approach of using SPIFFE helps to unify both non-human and human identity and delegation while eliminating static secrets/passwords or long-lived credentials. Another internet draft publication specifies automatically registering a client on first use.

In the next blog, we look at how to eliminate client secrets for authorization flows by authenticating to the Authorization Service with a SPIFFE SVID.

This is part of a much larger showcase of MCP / Agent2Agent identity, delegation, and authorization I'm working on. Please follow (@christianposta or /in/ceposta) along if interested.

Implementing MCP Dynamic Client Registration With SPIFFE and Keycloak was published on July 29, 2025.

YOU MIGHT ALSO ENJOY (VIEW ALL POSTS)

- Mitigate Prompt Injection Attacks With A2AS and Agentgateway
- Building an MCP Gateway with Apigee API Gateway
- Understanding Sessions in Agent to Agent Communication